

TR-10

AI Engineering & Machine Learning

FULL SYLLABUS

SKILLS & TOOLS YOU WILL MASTER



Python + scikit-learn



PyTorch + Transformers



LangChain + LangGraph



RAG + Pinecone



Agents + Mem0 + MCP



FastAPI + Docker

10 Reasons This Works.

Why students choose Saarathi – and how each card helps you ship a real career.

01

10 Max / Batch

Not 50. Not 30. Ten. Every student seen, heard, and unblocked – every class.

02

Sunday Open Classroom

Every Sunday the classroom is open. Come in, pair-program, get unstuck, or just focus.

03

Weekly Detailed Syllabus

Full syllabus from day one. Every week: what to read, what to build, what the outcome is.

04

Revision Every 5th Day

Every fifth class is a revision session. Nothing piles up or slips through.

05

Week 1: Mindset

The first week is about getting into the right headspace before touching a line of code.

06

3 Ways of Learning

Self-learning + collaboration + mentorship. All three built into every course.

07

1:1 Mock Interview

Real mock interview with your mentor in the final week. Actual hiring prep, not fluff.

08

CV + LinkedIn

Resume writing and LinkedIn optimisation. Leave with a profile that gets replies.

09

Real Projects

Every course ends with a deployed, publicly accessible project you can show employers.

10

Gate Assessment

Aptitude test before Day 1. Maps your strengths, weaknesses, learning style, and pace – then builds your personal plan.

saarathiacademy.com.np/ai-engineering-and-machine-learning-course-in-nepal

+977-9744442469 · +977-9761095364

AI Engineering & Machine Learning

OFFICIAL COURSE SYLLABUS

TR-10: AI ENGINEERING & MACHINE LEARNING COURSE IN NEPAL, LLMS, RAG, AGENTS, FINE-TUNING, MODEL DEPLOYMENT & ML

COURSE OVERVIEW

PARAMETER	DETAILS
Course Code	TR-10
Course Title	AI Engineering & Machine Learning Course in Nepal, LLMS, RAG, Agents, Fine-Tuning, Model Deployment & ML
Total Duration	12 Weeks (3 Months)
Weekly Schedule	Monday to Friday (5 Days/Week)
Daily Contact Hours	2 Hours (live) + 2 Hours (self-study expected)
Delivery Mode	Online + Offline
Offline Location	Old Baneshwor Chowk, Kathmandu, Nepal
Contact (Call/WhatsApp)	9744442469, 9761095364
Total Contact Hours	120 Hours
Maximum Batch Size	10 Students
Course Fee	NPR 39,750
Level	Beginner-friendly, taught from the fundamentals → AI Engineering Apprentice
Target Outcome	AI Engineering Apprentice / Junior LLM-App Developer, with a realistic 3–6 month runway into junior AI engineer roles

Table of Contents

COURSE OVERVIEW	2
WHY THIS COURSE	3
WHO IS THIS COURSE FOR	4
PREREQUISITES	5
YOUR LEARNING WEEK	5
PHASE STRUCTURE OVERVIEW	5
PHASE 1, PYTHON, MACHINE LEARNING & DEEP LEARNING FOUNDATIONS (Weeks 1–5, 50 Hours)	5
PHASE 2, LLMS, RAG & VECTOR DATABASES (Weeks 6–8, 30 Hours)	7
PHASE 3, AGENTS, EVALUATION & MODEL ADAPTATION (Weeks 9–11, 30 Hours)	13
PHASE 4, PRODUCTIONIZE, CAPSTONE & CAREER (Week 12, 10 Hours)	18
PROJECT POOL	23
TECHNICAL COMPETENCIES ACHIEVED	25
SOFT SKILLS DEVELOPED	26
JOB READINESS ASSESSMENT	28
ASSESSMENTS	28
CERTIFICATION REQUIREMENTS	28
RECOMMENDED BOOKS & RESOURCES	29
FINAL OUTCOMES	29
CAREER PATHS & TRAJECTORY	30
ENROLLMENT	30

WHY THIS COURSE

For people who want to become junior AI engineers shipping real LLM, RAG and agent systems, not ChatGPT wrappers. Employers want engineers who understand what an embedding is, can build an evaluated RAG pipeline, design tool-using agents and deploy models with production discipline. Those roles are in demand in Nepal and on remote international teams. Pay scales with your skills and portfolio; we don't promise a salary.

The program is foundations-first: a five-week base of Python from scratch, the numpy/pandas/SQL data toolkit, a right-sized classical machine-learning block, and deep-learning and transformer intuition. The metrics, train/validation/test discipline, cross-validation and leakage cures from that block later power your LLM evaluations. Only then do you move to LLMs, RAG, agents, fine-tuning and local inference.

The discipline employers screen for runs through every phase: design an evaluation, justify a retrieval choice, put a cost figure in your README, trace every model call, ship to a public URL. You leave with a deployed, evaluated, monitored capstone, not a notebook.

AI productivity tools (Cursor for coding, Claude Artifacts for prototyping) are woven in from Week 1.

WHO IS THIS COURSE FOR

- True beginners who want to start from Python basics and build a real AI engineering career
- Python-comfortable learners who want to move beyond tutorials into deployed AI systems
- Data scientists transitioning from notebook analysis into engineering and deployment
- Full stack or backend developers specialising into AI systems
- ML learners who want production discipline around what they already know
- Builders who want to ship real LLM + RAG + agent products instead of demos

PREREQUISITES

- No prior coding required, Python is taught from scratch in Weeks 1 and 2
- A laptop with 8GB+ RAM, 16GB is better, no GPU needed
- Comfortable using a computer daily, files and folders
- Basic English reading and writing, you will write short READMEs and progress updates
- Saarathi Gate Assessment before Day 1, diagnostic, no pass or fail

Nothing to install beforehand. VS Code, Python and the OpenAI, Anthropic and HuggingFace accounts are set up together in the first class.

Optional warm-up. None of this is required and Week 1 assumes zero, but arriving with it makes the first two weeks easier:

- 3Blue1Brown Neural Networks chapter 1 on YouTube, visual intuition only, no code
- The first 3 sections of freeCodeCamp Scientific Computing with Python at [freecodecamp.org/learn](https://www.freecodecamp.org/learn), variables, loops and functions
- Skim one AI product you already use and write down what you think it does under the hood, we come back to it in Week 6

YOUR LEARNING WEEK

DAY	ACTIVITY
Mon–Fri	2-hour live class with live coding and system walkthroughs
Mon–Fri	2-hour self-study (notebook practice, reading papers)
Sunday	Open classroom or rest
Day 5 (every week)	Last hour of the session is revision and architecture review

PHASE STRUCTURE OVERVIEW

PHASE	TITLE	DURATION	FOCUS
Phase 1	Python, Machine Learning & Deep Learning Foundations	5 Weeks (50 Hours)	Python from scratch, the numpy/pandas/SQL data toolkit, a right-sized classical ML block (metrics, cross-validation, leakage cures, trees), then a dedicated deep-learning & transformer-intuition week
Phase 2	LLMs, RAG & Vector Databases	3 Weeks (30 Hours)	Prompting, context engineering, multi-provider APIs, embeddings, vector databases, evaluated RAG with reranking
Phase 3	Agents, Evaluation & Model Adaptation	3 Weeks (30 Hours)	Tool-using agents, LangGraph orchestration, MCP, memory layers, eval & safety (OWASP LLM Top 10), fine-tuning & local inference
Phase 4	Productionize, Capstone & Career	1 Week (10 Hours)	Ship the capstone live (FastAPI, Docker, tracing, cost dashboard), demo day, portfolio, resume, mock interviews

PHASE 1, PYTHON, MACHINE LEARNING & DEEP LEARNING FOUNDATIONS (Weeks 1–5, 50 Hours)

- **Duration:** 5 Weeks (50 Hours)
- **Phase Objective:** By the end of this phase you can write typed, error-handled Python, wrangle real datasets with numpy and pandas, pull data with SQL, train and honestly evaluate a scikit-learn baseline, and explain what a transformer is doing well enough to debug it later. This is the deliberately unhurried base: five full weeks before any LLM work. Python the language is taught from scratch across Weeks 1–2 so true beginners build a real foundation instead of memorising syntax. Week 1 stays concrete (values, control flow, collections) with no hidden abstractions; only after a full week of writing real code do you meet functions, classes, type hints and error handling, after which numpy and pandas begin the data toolkit. The ordering is deliberate: language fundamentals before data libraries, the data toolkit before models, and the eval discipline of classical ML (train/validation/test, cross-validation, leakage) before any LLM, because that exact discipline is what you reuse to evaluate RAG and agents later. Every LLM, RAG and agent layer downstream is still ordinary Python calling APIs and handling data; skip the base and the higher layers become guesswork. This phase is what separates an engineer who can reason about a model from someone who can only paste prompts.

Week 1, Python Foundations I – Core Language & Tooling

- **Weekly Hours:** 10 Hours
- **Theme:** Start from absolute zero and get fluent with the concrete building blocks, values, control flow and collections, with every line producing a visible result and no hidden abstractions yet.
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 1 Topics Covered

TOPIC	DETAILS
Development Environment & Git	VS Code installed, the terminal, and Git basics (<code>init</code> , <code>add</code> , <code>commit</code> , <code>push</code>) so your code is version-controlled from your very first script. Scope: Core, hands-on. Docker is in Week 12.
Values, Variables, Types & Operators	<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code> and <code>None</code> , and why a variable is just a name pointing at a value in memory; arithmetic, comparison and boolean operators, and which values count as truthy or falsy. Scope: Core, covered in full.
Strings & f-strings	Indexing, slicing and formatting text with f-strings, the format you will use for every prompt and log line later. Scope: Core, hands-on.
Control Flow	<code>if</code> / <code>elif</code> / <code>else</code> branching, building the guard-clause-then-happy-path habit. Scope: Core, covered in full.
Loops	<code>for</code> and <code>while</code> , <code>range</code> , <code>break</code> and <code>continue</code> , and the accumulate-in-a-loop pattern. Scope: Core, covered in full.

TOPIC	DETAILS
Collections – Lists, Tuples, Dicts & Sets	Ordered lists and tuples (when a mutable list beats a fixed tuple, zero-based indexing and exclusive slice ends), then labelled key→value dictionaries and duplicate-free sets, and why a dict's constant-time lookup beats scanning a parallel list. Scope: Core, covered in full.
AI Pair-Coding, First Look	Cursor autocomplete on from the start, suggesting code as you type; reading and judging that code comes in Week 2. Scope: Awareness; verify-discipline drilled in Week 2.

Weekly revision (last hour): Read-aloud and predict-the-output drills on variables, control flow, loops and collections; a broken-then-fixed walkthrough of a zero-based-indexing bug; Q&A for the week's doubts; preview of Week 2, functions, classes and the move into engineering Python.

Week 2, Python Foundations II – Engineering for AI

- **Weekly Hours:** 10 Hours
- **Theme:** Finish the Python language early in the week, learn to structure code like an engineer (functions, classes, type hints, errors and file I/O); the rest of the week then meets numpy and pandas as the on-ramp into the data toolkit.
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 2 Topics Covered

TOPIC	DETAILS
Functions & Comprehensions	Positional and keyword arguments, defaults, <code>*args</code> / <code>**kwargs</code> , return values and scope; extract a function from last week's code so the abstraction clicks, never a mutable default like <code>def f(x=[])</code> . Then list and dict comprehensions, read as sugar over a <code>for</code> loop you already understand. Scope: Core, covered in full.
Type Hints	Annotate signatures with <code>str</code> , <code>int</code> , <code>list[str]</code> , <code>dict[str, int]</code> , <code>Optional</code> and return types so Pydantic and your editor catch mistakes; a hint is a contract, not enforced at runtime. Scope: Core, hands-on.
Error Handling	<code>try</code> / <code>except</code> / <code>else</code> / <code>finally</code> , raising informative errors and one custom exception class. No bare <code>except</code> . Scope: Core, covered in full.
File & JSON I/O	Reading and writing text and JSON with <code>open()</code> , context managers and <code>pathlib</code> , always with explicit encoding. Scope: Core, hands-on.
Modules, Imports & Lean OOP	A script split into your own modules, <code>import</code> -ed, then others installed and imported, with the <code>if __name__ == "__main__"</code> guard; then classes with <code>__init__</code> , <code>self</code> , instance methods and one inheritance example, and when

TOPIC	DETAILS
	a class beats a bag of dicts. No metaclasses or ABCs. Scope: Hands-on; OOP basics only.
numpy & pandas, First Look	numpy arrays and vectorised math, then pandas DataFrames (read a CSV, select, filter), the on-ramp into next week's data toolkit. Scope: Hands-on intro; deepened in Week 3.
Disciplined AI Pair-Coding	Cursor and Copilot for autocomplete, explanation and refactoring, every line read and verified before accepting; never paste keys or data into a chat. <code>async / await / asyncio.run</code> met as awareness only. Scope: Core workflow; <code>async</code> drilled in Week 6.

Weekly revision (last hour): Review of functions, comprehensions, type hints, error handling, file I/O, modules and lean OOP, plus the numpy/pandas first look; broken-then-fixed drills on mutable default arguments, bare `except` and `==` vs `is`; Q&A; preview of Week 3, the full data toolkit and supervised machine learning.

Week 3, Data Toolkit & Supervised Machine Learning

- **Weekly Hours:** 10 Hours
- **Theme:** Finish the numpy/pandas/SQL/HuggingFace data toolkit, then learn supervised machine learning in one honest beginner arc: split the data, fit your first models, measure them, catch overfitting, cross-validate, and kill data leakage with a Pipeline. This is the eval-discipline week, and that discipline is exactly what you reuse to evaluate LLMs later.
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 3 Topics Covered

TOPIC	DETAILS
Data Toolkit, Finished + SQL	<code>venv</code> , <code>pip</code> , pinned <code>requirements.txt</code> and Jupyter ; numpy broadcasting and boolean masks; pandas <code>.loc</code> / <code>.iloc</code> , filtering, <code>groupby</code> and <code>merge</code> , building on the Week 2 first look; then querying SQLite and Postgres with <code>SELECT</code> , <code>WHERE</code> , <code>JOIN</code> , <code>GROUP BY</code> and aggregates, reading results straight into pandas . Scope: Core, hands-on. Spark and Dask at awareness only.
HuggingFace Datasets & Splits	Loading datasets with <code>load_dataset</code> , <code>select</code> , <code>filter</code> , <code>map</code> , and making reproducible train/validation/test splits. A split grades the model on data it never saw. Scope: Hands-on.
Supervised Learning, Regression & Classification	Linear regression to predict a number (scaled-coefficient interpretation, RMSE, MAE and R^2 ; Ridge/Lasso at awareness), then logistic regression for yes/no outcomes with threshold tuning, <code>class_weight</code> , and the confusion matrix, precision, recall, F1 and ROC-AUC. Scope: Covered in full; PR-AUC at awareness.

TOPIC	DETAILS
Overfitting & Cross-Validation	Learning curves for a model that memorises noise versus one too simple, then a stratified k-fold pass; report mean and spread, not one lucky split. Scope: Core, covered in full.
Data Leakage & the Pipeline Cure	Data leakage is test-set or answer-key info sneaking into training (e.g. scaling before splitting); one visceral cheating-model demo, then the cure, chaining <code>StandardScaler</code> / <code>OneHotEncoder</code> and the model in a scikit-learn Pipeline . Scope: Core, covered in full.
Why This Powers Your LLM Work	The same metrics, held-out splits and no-leakage habit become your RAG eval set, your LLM-as-judge rubric and your agent-evaluation discipline in Phase 2–3. Scope: Concept bridge, made explicit.
AI-Assisted ML	Copilot and Claude Artifacts for drafting scikit-learn code, with dtypes, splits and metrics verified after. Never paste keys or data into a chat. Scope: Hands-on.

Weekly revision (last hour): Review of the data toolkit, train/test discipline, regression and classification metrics, overfitting, cross-validation and the leakage-curing Pipeline; the imbalanced-metric trap (why accuracy lies on a 99%-negative dataset); Q&A; preview of Week 4, tree models, clustering and feature engineering.

Week 4, Tree Models, Clustering & Feature Engineering

- **Weekly Hours:** 10 Hours
- **Theme:** Round out classical machine learning, right-sized for an AI engineer: one trees-vs-forest lab, a light clustering demo and real feature engineering, then draw the explicit line from features to the chunking and context decisions you will make in RAG. No deep learning yet, by design, so the week stays calm and consolidates the base.
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 4 Topics Covered

TOPIC	DETAILS
Trees & Random Forests	<code>DecisionTreeClassifier</code> for readable rules and <code>RandomForestClassifier</code> for accuracy and feature importance, with the few hyperparameters that matter (max depth, n_estimators, min_samples_leaf). Scope: One comparison lab; deep tuning and XGBoost/LightGBM at awareness.
Clustering	One light k-means demo (elbow method, silhouette score, scale first) to see unsupervised grouping. Scope: One hands-on demo; awareness beyond.

TOPIC	DETAILS
Feature Engineering & ColumnTransformer	Engineered ratios, bins and date-part features, then <code>StandardScaler</code> / <code>OneHotEncoder</code> / <code>ColumnTransformer</code> and the model chained in one leakage-safe <code>Pipeline</code> . Every transform lives in the Pipeline . Scope: Core, covered in full.
Features → Chunking Bridge	The judgement you build choosing and shaping features is the same one you will use choosing chunk size, overlap and context for retrieval; this is the explicit hand-off from classical ML to RAG. Scope: Concept bridge, made explicit.
Model Comparison & Interpretation	The tuned random forest against the Week 3 logistic-regression baseline on the same cross-validated metrics, weighing accuracy versus interpretability. Scope: Core, covered in full.
When a Simple Model Beats an LLM	The cost, latency and reliability cases where a logistic regression, a rule or a small classifier beats calling an LLM, a judgement employers actively test. Scope: Concept, covered in full.
AI-Assisted ML	Copilot and Claude Artifacts for drafting tree, clustering and feature code, with splits, leakage and metrics verified after. Never paste keys or data into a chat. Scope: Hands-on.

Weekly revision (last hour): Review of decision trees and random forests, feature importance, the k-means demo, the `ColumnTransformer` Pipeline, the features-to-chunking bridge and the simple-model-vs-LLM judgement; Q&A; preview of Week 5, deep learning and transformer intuition.

Week 5, Deep Learning & Transformer Intuition

- **Weekly Hours:** 10 Hours
- **Theme:** Build enough deep-learning and transformer intuition to debug LLMs later: conceptual first, code second, with a whole week to absorb it. This is the bridge between classical ML and the LLM work that starts in Week 6.
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 5 Topics Covered

TOPIC	DETAILS
Neural-Network Intuition	How neurons, layers and activations like ReLU and sigmoid learn patterns. Visual first, calculus deferred. Scope: Intuition only.

TOPIC	DETAILS
PyTorch Fundamentals	PyTorch tensors and <code>autograd</code> , then read and run one training loop on a toy dataset with a logged loss curve, SGD vs Adam. A guided build, every line understood. Scope: The one deep hands-on build of the week.
Attention Intuition	Each word looks around the sentence and weights who to listen to; then name the query, key and value vectors and why Attention replaced RNNs. Scope: Diagram and intuition only, no code.
Transformer Anatomy	Encoder-only (BERT-class), decoder-only (GPT-class) and encoder-decoder (T5-class) told apart so you pick the right family. Tokenisation and Embeddings introduced, and why they make RAG possible. Scope: Intuition only, no code.
HuggingFace Transformers	Any pretrained checkpoint loaded with <code>pipeline</code> , <code>AutoModel</code> and <code>AutoTokenizer</code> , with inference in a few lines. Scope: Hands-on.
Fine-Tune Walkthrough (Week-11 preview)	A short guided walkthrough of a <code>Trainer</code> fine-tune notebook. Scope: Read-along preview only; built in Week 11.
CNN & RNN Awareness	Where convolutional and recurrent networks still ship (pre-transformer vision, on-device speech), so you recognise them in the wild. Scope: Awareness only, one line.

Weekly revision (last hour): Review of neural-net intuition, the PyTorch training loop, attention concepts and transformer anatomy, plus the HuggingFace inference run; Q&A session for clarifying doubts from the week; Preview of Week 6: Prompt engineering and LLM APIs

Phase 1 Project: Python-to-Transformer Foundations Notebook

Description: Ship a reproducible notebook demonstrating Python fundamentals, classical machine-learning fluency and transformer intuition, the foundation the rest of the course builds on.

Requirements:

- A Git-tracked Python CLI script reading a JSON file, transforming data and writing output (optionally calling a public API as a stretch), fully type-hinted
- A pandas data cleaning pipeline on a real dataset with a HuggingFace split for downstream ML
- A classical ML baseline (a scikit-learn Pipeline with a classifier or regressor, plus a tree/forest feature-importance read-out) evaluated with cross-validation and the right metrics
- A PyTorch training loop on a toy problem with logged loss curves (Adam vs SGD)
- An annotated transformer architecture diagram with a HuggingFace inference run (the hands-on fine-tune is built and evaluated in Week 11)

PHASE 2, LLMS, RAG & VECTOR DATABASES (Weeks 6–8, 30 Hours)

- **Duration:** 3 Weeks (30 Hours)
- **Phase Objective:** By the end of this phase you can drive LLM APIs across multiple providers with reliable structured outputs, build and benchmark an embedding-based retrieval layer, and ship a RAG pipeline whose quality you can actually measure with RAGAS. Prompting and APIs come first because retrieval has nothing to generate against until you can call a model well; embeddings and vector databases come next because RAG is mostly a retrieval problem; evaluation comes last because an unmeasured pipeline is a demo, not a system. This is the phase where employer-facing work begins, a cited, evaluated RAG app over private documents is the single most common junior AI engineering deliverable.

Week 6, Prompt Engineering & LLM APIs

- **Weekly Hours:** 10 Hours
- **Theme:** Use LLM APIs like an engineer, not a prompt-twiddler.
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 6 Topics Covered

TOPIC	DETAILS
LLM Lifecycle	How a model goes from pretraining through supervised fine-tuning, RLHF and alignment, enough to reason about model behaviour. Scope: Intuition only.
Prompt Patterns	Zero-shot, few-shot and chain-of-thought prompting, plus the style reasoning models need: ChatGPT Thinking, Claude Extended Thinking, Gemini Pro thinking and DeepSeek R1 / V-series. Scope: Core, covered in full.
Context Engineering	Curating every token the model sees at inference (system prompt, tool definitions, tool results, retrieved chunks, memory, history) so a finite window holds what the task needs. The window is a small desk, not a filing cabinet. Scope: Covered in full; agent-side techniques drilled in Week 9.
Structured Outputs	Valid JSON forced with OpenAI Structured Outputs strict mode and Anthropic forced tool use, JSON-mode as fallback. Strict schemas over regex parsing. Scope: Hands-on across providers.
OpenAI SDK & Anthropic SDK	OpenAI chat completions, message roles (system/user/assistant), token streaming, and production resilience, handling timeouts, backing off on 429 rate limits with exponential backoff and retrying idempotently (a <code>tenacity</code> -style retry decorator); Anthropic Messages API with system prompts and prompt caching via <code>cache_control</code> , plus cost intuition for cached vs cold calls. Scope: Hands-on across both provider SDKs.
Secrets & Config	API keys in environment variables loaded with <code>python-dotenv</code> , a <code>.env</code> that is git-ignored and never committed, and a checked-in <code>.env.example</code> template, the secrets habit every deployed app needs. Scope: Hands-on.

TOPIC	DETAILS
Provider Choice & Groq	A decision matrix across OpenAI (ChatGPT) , Claude , Google Gemini (Pro/Flash) , reasoning and open-weight models (DeepSeek , Qwen , Llama) on task fit, cost, latency and context-window length, with Groq 's ultra-low-latency inference wired in as a fallback tier behind your primary provider. Scope: Core, covered in full; Groq hands-on as the fallback tier.
AI Gateway Awareness	OpenRouter , Portkey and Helicone as routing layers for retries, failover and spend tracking. Scope: Awareness only; demo returns in Week 12.
Streaming UI & Async	Token streaming into a minimal chat surface with <code>async / await</code> against real API calls (the Week-2 async intro becomes hands-on here, motivated by a real network wait), with cancel-on-unmount so abandoned requests stop billing. Scope: Hands-on.

Weekly revision (last hour): Review of LLM architecture, prompting strategies, context engineering, structured outputs and multi-provider setup; Q&A session for clarifying doubts from the week; Preview of Week 7: Embeddings and vector databases

Week 7, Embeddings & Vector Databases

- **Weekly Hours:** 10 Hours
- **Theme:** Retrieval is 80% of RAG, spend the time here.
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 7 Topics Covered

TOPIC	DETAILS
Embeddings Core	Vectors, cosine similarity and dimensionality as the maths behind semantic search. Intuition first. Scope: Core, covered in full.
Modern Embedding Models	The real skill is choosing an embedding model by language coverage, cost and quality, tested against a small labelled retrieval set, not by reputation. Compare two or three across the field, a hosted API (OpenAI text-embedding-3), a strong multilingual model with Nepali (BAAI bge-m3) and an open-weight local option (HuggingFace sentence-transformers); know that Matryoshka embeddings let you trade dimensions for cost. Scope: Core, covered in full.
Pinecone	A managed Pinecone vector index, document upserts with metadata, filtering by tenant or category, and namespace-strategy design. Scope: Hands-on.
Local and In-Database Stores	ChromaDB on a laptop for dev, <code>pgvector</code> inside Postgres for production. Scope: ChromaDB and pgvector hands-on; Qdrant , Weaviate , Milvus at awareness.

TOPIC	DETAILS
Chunking Strategies	Fixed-token, semantic and parent-document chunking, and how chunk size and overlap trade recall against precision. recall@k, MRR and nDCG are the numbers you move. Scope: Core, covered in full.
Hybrid Search	Dense embedding search combined with sparse BM25 , then a Cohere Rerank or bge-reranker stage. Reranking is non-optional in production. Scope: Core, covered in full.
Multi-Modal Inputs	Images sent with text using OpenAI vision (ChatGPT), Claude Vision and Gemini vision (the long-context multimodal option) in base64 and URL forms. Scope: Hands-on.
Realtime and Voice Awareness	OpenAI Realtime and ElevenLabs for speech-in / speech-out. Scope: Awareness only.

Weekly revision (last hour): Review of embeddings, vector DBs, ChromaDB, pgvector, hybrid retrieval and multi-modal inputs; Q&A session for clarifying doubts from the week; Preview of Week 8: RAG pipeline development and evaluation

Week 8, RAG Pipeline Development

- **Weekly Hours:** 10 Hours
- **Theme:** Build an evaluated RAG pipeline, not a demo.
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 8 Topics Covered

TOPIC	DETAILS
LangChain and LCEL	RAG chains composed as small typed runnables with LangChain Expression Language (LCEL). Current as of LangChain 1.0. Scope: Core, covered in full; stateful/agentive moves to LangGraph in Week 9; AgentExecutor now legacy/maintenance.
End-to-End RAG	The full pipeline (ingest, chunk, embed, store, retrieve, rerank, answer with citations) built over your own documents. Scope: Hands-on.
Advanced Retrieval	Recall improved with multi-query rewriting, parent-document retrieval and query decomposition for multi-fact questions. Scope: Hands-on.
LlamaIndex Alternative	LlamaIndex query engines and document-graph index types; pick it when graph features beat LangChain 's plumbing. Scope: Hands-on.

TOPIC	DETAILS
RAGAS Evaluation	RAG scored with faithfulness, context precision, context recall and answer relevancy on a real eval set. Ship the eval set before the bot. Scope: Core, covered in full.
Beyond Basic RAG - Agentic & GraphRAG	Agentic RAG , where an agent decides when to retrieve, what to query and when it has enough instead of a fixed retrieve-then-answer chain (built in Week 9); and graph-augmented retrieval for multi-hop questions with Microsoft GraphRAG , LazyGraphRAG , LightRAG and Neo4j as backend, where graph indexing costs roughly 10-40× vector RAG, so route simple queries to vector RAG. Scope: Awareness only.
Hosted Agent APIs	The OpenAI Responses API plus Conversations API against a custom RAG build on control, cost and lock-in. Scope: Awareness only; Assistants API removed 26 August 2026.
Production Concerns	A per-stage latency budget, caching of responses and embeddings, and per-request cost estimation. Scope: Hands-on.

Weekly revision (last hour): Review of LangChain (LCEL, with LangGraph as the stateful path), basic RAG, retrieval upgrades, LlamaIndex, RAGAS, agentic-vs-graph RAG awareness and Responses/Conversations-API-vs-custom-RAG tradeoffs; **Capstone kickoff in this architecture-review slot:** pick your problem, dataset, success metric and stakeholder and secure mentor sign-off (capstone must be reproducible end-to-end on the provided laptop tier); Q&A session for clarifying doubts from the week; Preview of Week 9: Tool-using agents and LangGraph orchestration

Phase 2 Project: RAG Application over Private Docs

Description: Ship a RAG application over a private document corpus with hybrid retrieval, retrieval upgrades and measurable quality scored with RAGAS.

Requirements:

- A document ingestion + chunking pipeline with a justified chunking strategy
- A vector index (Pinecone or Chroma) plus BM25 hybrid retrieval
- An LCEL-based RAG pipeline returning grounded, cited answers
- RAGAS evaluation on faithfulness and answer relevancy with a debug playbook

PHASE 3, AGENTS, EVALUATION & MODEL ADAPTATION (Weeks 9–11, 30 Hours)

- **Duration:** 3 Weeks (30 Hours)
- **Phase Objective:** By the end of this phase you understand the agent harness (the loop-and-plumbing layer around the model), can build a tool-using agent that calls real tools safely and orchestrate it as a LangGraph workflow with branching, supervision, human-in-the-loop gates and checkpointing, prove it is safe and good through evaluation and red-teaming, and adapt models through fine-tuning or local inference when prompting and RAG are not enough. Agents and their orchestration share one intensive week now: you are past the deliberately gentle foundations weeks and ready for the step up in pace. Evaluation comes next because you need a system worth evaluating, and model adaptation comes last because it is the most expensive lever and should only be reached for once you can show prompting and RAG fall short. This is the phase where tool-using, evaluated, safety-gated agents, exactly the systems that distinguish a hireable AI engineer from a prompt wrapper, come together.

Week 9, Tool-Using Agents, Harnesses & LangGraph Orchestration

- **Weekly Hours:** 10 Hours
- **Theme:** Make an agent use real tools reliably, remember across sessions and stay inside its context window, then orchestrate it as a LangGraph workflow that branches, supervises sub-agents, pauses for human approval and survives restarts. The spine of the week is the **agent harness**, the loop-and-plumbing layer wrapped around the model: you build a minimal one by hand so the concept is concrete, then see production harnesses (the Claude Agent SDK, Deep Agents) and the LangGraph runtime do the same job at scale. (Capstone was scoped in the Week 8 architecture-review slot; this is the first week of the deliberately steeper second half, now that the foundations weeks are behind you.)
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 9 Topics Covered

TOPIC	DETAILS
Agent Harness & the Agentic Loop	The harness is the orchestration layer that turns a raw LLM into a runnable agent, the model decides, the harness does : the model only emits text and tool-call requests, while the harness executes the tools, manages files and context, gates dangerous actions and loops. Build a minimal harness yourself (the agentic loop, gather context → act → verify → repeat, ending when the model returns no tool call) over three atomic tools (<code>read_file</code> , <code>write_file</code> , <code>run_shell</code>), then see the same shape in production through the Claude Agent SDK (the Claude Code harness exposed as a library). Tell a harness (batteries-included, e.g. Claude Code) from a framework (components you assemble, e.g. LangChain) from a runtime (the durable execution environment the agent runs in). Scope: Hands-on; the hand-built loop is the spine the rest of the week hangs on.

TOPIC	DETAILS
Agent Anatomy & Tools	The ReAct loop and when an agent genuinely beats a deterministic chain, and when it does not (prefer a deterministic pipeline unless the control flow is truly open-ended, the judgement employers test); typed Pydantic tool schemas, tool errors surfaced back to the model, and the loop run with LangChain's <code>create_agent</code> (LangGraph engine, retries and step limits). Scope: Hands-on; the older <code>AgentExecutor</code> and <code>create_react_agent</code> are now legacy (maintenance mode), not the path you build on.
Memory & Agentic Memory Layers	Short-term window memory vs long-term recall across sessions; wire a real memory layer with Mem0 . Scope: Hands-on with Mem0 ; Zep (Graphiti) , Letta (MemGPT) , LangMem at awareness.
Context Engineering for Agents & Deep Agents	Keep a long-running agent inside its window with compaction, note-taking, sub-agent isolation, just-in-time retrieval and tool-result clearing; then meet Deep Agents (deepagents) , the batteries-included harness that productises exactly these techniques through four pillars, a <code>write_todos</code> planning tool, task sub-agents with isolated context, a virtual file system that offloads large outputs to disk, and a detailed system prompt. Build a <code>create_deep_agent</code> research agent (you supply one web-search tool; the planner, file system and sub-agents are injected), and learn when a deep agent beats a plain ReAct agent (long-horizon work, planning, context offload) versus when a hand-built LangGraph StateGraph is the right call instead (a custom control-flow topology). Scope: Hands-on with deepagents (built on LangGraph , pre-1.0 and fast-moving); LLMLingua-2 and lean-ctx an optional demo.
MCP, the Agent Protocol Stack & Runtimes	A minimal MCP server exposing one typed tool, consumed from Claude Desktop or Cursor ; MCP is now an open standard under the Linux Foundation (Agentic AI Foundation), adopted across Claude, ChatGPT, Gemini, Cursor and Copilot, so build to the spec, not one vendor. Place it in the wider protocol stack, MCP (agenttools), A2A (agentagent) and AG-UI (agentuser); trace with LangSmith ; and compare the agent runtimes/harnesses, LangGraph vs the OpenAI Agents SDK (framework) vs the Claude Agent SDK (harness). Scope: Hands-on MCP + LangSmith; A2A , AG-UI , CrewAI , AutoGen , Computer Use at awareness.
LangGraph Orchestration - StateGraph, Supervisor, HITL & Checkpointing	The workflow modelled as nodes, edges and typed shared state (one dictionary every node reads and writes) in LangGraph , routed on tool results or user input with guard rails against runaway loops; one orchestrator node dispatching to specialist sub-agents; interrupt-and-approve checkpoints before any irreversible action (payments, deletes, messages); and graph state saved to durable storage so workflows resume across restarts without replaying side-effecting nodes, with retry on flaky tools. Scope: Hands-on.
Hugging Face Agents Certification (graded, self-study)	The free HuggingFace Agents Course and its Certificate of Fundamentals (Unit 1 + quiz), claimed by end of Week 10. Scope: Graded self-study; full certificate an optional stretch.

Weekly revision (last hour): Review of the agent harness and the agentic loop (your hand-built minimal harness, the model-decides/harness-does split), ReAct, tool design and the `create_agent` loop, agentic memory (Mem0 + the Zep/Graphiti/Letta/LangMem landscape), agent context engineering and **Deep Agents** (the four pillars), the MCP build and the MCP/A2A/AG-UI protocol stack, LangSmith and the LangGraph orchestration layer (branching, supervisor, HITL, checkpointing), plus a progress check on the Hugging Face Agents certification milestone; Q&A session for clarifying doubts from the week; Preview of Week 10: Evaluation and safety (including the eval harness, the other sense of "harness")

Week 10, Evaluation & Safety

- **Weekly Hours:** 10 Hours
- **Theme:** Make the capstone provably good and safe: LLM-as-judge, agent evaluation and a prompt-injection guard.
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 10 Topics Covered

TOPIC	DETAILS
LLM Evaluation Pipelines	Golden eval sets (fixed, trusted test questions with known answers) and LLM-as-judge graders with disclosed rubrics. Don't judge a model with itself where avoidable. Scope: Core, covered in full.
Agent Evaluation	Task success rate, tool-call correctness and trajectory quality measured on a held-out task set. Lock metrics before scaling tools. Scope: Core, covered in full.
Prompt Injection, OWASP LLM & Agentic Top 10	Defences against direct Prompt injection in user input and indirect injection in RAG-poisoned documents with tool allowlists, untrusted-content isolation and system-prompt-echo stripping, plus the OWASP LLM Top 10 risks, especially LLM01 Prompt Injection and LLM06 Excessive Agency, and the newer OWASP Top 10 for Agentic Applications (2026) for tool-using agents (agent goal hijacking, tool misuse, memory/context poisoning). Scope: Hands-on; Guardrails AI, NeMo Guardrails at awareness.
Eval Frameworks & the Eval Harness	The landscape, a default pick, and one eval gate wired into CI; an eval harness is the rig that runs your system against a fixed test set and scores it (this is the evaluation sense of "harness", distinct from the agent harness you built in Week 9); every LLM call traced as spans (a trace is the whole request, a span is one step, a retrieval or a model call) so you can debug failures and watch drift. Scope: Awareness; default RAGAS + Promptfoo + DeepEval, LangSmith + Langfuse over OpenTelemetry GenAI; Braintrust, OpenAI Evals, Helicone, Arize Phoenix as pointers.
Safety Writeup	One writeup: eval metrics sliced by segment for bias, the model card (intended use, limitations, scores) and a red-team sweep, mentor-signed. Scope: Hands-on.

Weekly revision (last hour): Review of LLM-as-judge, agent evaluation, prompt injection, the OWASP LLM Top 10 and the OWASP Agentic Top 10, and the consolidated safety writeup, plus collection of the Hugging Face Agents Certificate of Fundamentals milestone (due this week); Q&A session for clarifying doubts from the week; Preview of Week 11: Fine-tuning and local inference

Week 11, Fine-Tuning & Local Inference

- **Weekly Hours:** 10 Hours
- **Theme:** Adapt models and run them locally.
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 11 Topics Covered

TOPIC	DETAILS
Fine-Tune vs Prompt	The decision rule: only fine-tune after prompting and RAG plateau, and only with a labelled dataset and eval set in place. Scope: Core, covered in full; no pretraining from scratch.
Dataset Preparation	Instruction-format pairs assembled, deduplicated and split train/val; synthetic data from stronger-model distillation with licence checks. Scope: Hands-on.
OpenAI Fine-Tuning API (graded path)	The hosted OpenAI fine-tune service end-to-end, its data format and cost per token, and evaluation against the base model. Scope: Graded hands-on.
Ollama and Local Serving (graded path)	A quantised open-weight model pulled, run and served locally with an OpenAI-compatible endpoint, called from Python. Scope: Graded hands-on.
Quantisation and GGUF	Quantisation shrinks stored numbers to fit RAM at a small accuracy cost; compare Q4, Q5 and Q8 against quality and footprint to fit 8GB hardware. Scope: Core, covered in full.
LoRA / QLoRA / PEFT (graded path)	A LoRA and 4-bit QLoRA adapter trained with HuggingFace PEFT and evaluated against the base model. Every student ships a hosted fine-tune and an open-weight LoRA adapter. Scope: Graded hands-on; runs on a free Colab T4 GPU.
Fully Local RAG	An end-to-end RAG stack with no external API key, deployable behind a corporate firewall. Scope: Optional stretch.
Production Self-Host & RL Awareness	vLLM and SGLang as the production serving stack, with continuous batching and paged attention for high-throughput multi-tenant serving (HuggingFace TGI is now in maintenance/legacy as of 2026); plus RLHF and DPO conceptually, and why most teams consume tuned base models. Scope: Awareness only.

Weekly revision (last hour): Review of the fine-tune decision rule, the hosted OpenAI fine-tune and the graded LoRA/QLoRA Colab T4 fine-tune, Ollama local serving and quantisation; Q&A session for clarifying doubts from the week; Preview of Week 12: Productionize, ship the capstone and career

Phase 3 Project: Evaluated Tool-Using Agent

Description: Ship a LangGraph tool-using agent wired with memory, evaluation and safety layers, plus a fine-tuned or locally-served variant that demonstrates model-adaptation fluency.

Requirements:

- A LangGraph agent with tools, persistent memory via a memory layer (Mem0) and LangSmith traces, with at least one agent context-engineering technique applied (compaction, just-in-time retrieval, tool-result clearing, or a Deep Agent's planning / sub-agent / file-system pillars)
 - A supervisor pattern plus a human-in-the-loop approval gate on destructive actions, with at least one tool exposed through a minimal MCP server
 - A short writeup of your agent's harness: the agentic loop, how tools are dispatched, and exactly where dangerous actions are gated
 - An evaluation pipeline with LLM-as-judge scoring plus input/output guardrails
 - A fine-tuning (OpenAI or LoRA/QLoRA) or local-inference (Ollama/GGUF) adaptation variant
-

PHASE 4, PRODUCTIONIZE, CAPSTONE & CAREER (Week 12, 10 Hours)

- **Duration:** 1 Week (10 Hours)
- **Phase Objective:** By the end of this final week your capstone is a deployed, demoable AI service, served behind FastAPI, containerised with Docker, traced and cost-tracked, reachable at a public URL, and you have packaged that work into a repo, resume and interview narrative a recruiter takes seriously. The heavy engineering is already done: you built the system across Weeks 6–11, wired its evaluation and safety layer in Week 10 and a fine-tuned or locally-served variant in Week 11, so this week is about shipping it for real and converting twelve weeks of work into a hireable story. Getting the system operable comes before the career polish because the hiring story is only credible once the project runs somewhere other than your laptop. Deeper MLOps tooling (MLflow experiment tracking, CI evals on every pull request, GPU-cloud deployment, AI-Gateway cost dashboards) is introduced as awareness and layered on as time allows, not mastered, because for a junior AI-engineer role the bar is "ship an API, trace it, watch its cost," not cluster operations.

Week 12, Productionize, Ship the Capstone & Career

- **Weekly Hours:** 10 Hours
- **Theme:** Ship the capstone live and land the interviews. Productionize and deploy first, then finalise the portfolio and run hiring prep. The DSA strand carried as self-study across Weeks 9–12 closes here.
- **Depth Focus:** Core classroom topics and required outcomes listed in this week's schedule
- **Awareness Focus:** Optional extension topics and stretch practice moved into the hands-on column

Week 12 Topics Covered

TOPIC	DETAILS
FastAPI Serving	Your capstone wrapped behind a typed HTTP API with Pydantic schemas, async endpoints and healthcheck plus version routes. Scope: Core, covered in full.
Docker & Live Deploy	Containerisation with a multi-stage Docker build and a slim base image, then deployment to a publicly reachable URL (a small cloud host, or a guided GPU-cloud deploy to RunPod / Lambda Labs as time allows). Any publicly reachable deployment qualifies. Scope: Docker + deploy hands-on; GPU-cloud as time allows; LangGraph Platform at awareness.
Tracing, Monitoring & Cost	Every request traced (latency, tokens, cost) with alerts on error rate and output drift at thresholds set before launch; watch p50/p95/p99, not an average. Put a real cost-per-request figure in your README. Scope: Core, hands-on.
Deeper MLOps (as time allows)	MLflow experiment tracking and model promotion (rollback awareness), CI evals (RAGAS / Promptfoo / LLM-as-judge on every pull request, because a prompt change is a code change), and an AI-Gateway cost dashboard with semantic / TTL caching. Scope: Awareness with short demos; layered on as time allows.
Capstone Finalisation	Your capstone's failure modes, architecture diagram and eval scorecard walked with your mentor for sign-off, then a README on every repo with the canonical

TOPIC	DETAILS
	template (problem / approach / eval results / cost / limits / run instructions), a clean Git history using feature branches and pull requests (the team workflow recruiters expect), a demo video and your strongest repos pinned. Scope: Core, hands-on.
Resume, LinkedIn & Remote Readiness	A CV rewritten with an AI-engineer headline, quantified bullets (recall@k, eval scores, cost-per-request) and Nepal-plus-remote variants; LinkedIn recruiter discovery on; never claim unmeasured "100% accuracy". Practise the async written-update habit and clear English that distributed teams gate on. Scope: Hands-on.
System-Design, Mock Interviews & DSA Drill	Timed whiteboard cases (RAG-at-scale, multi-tenant agent platform, cost-controlled LLM service) graded against a rubric, then a coding round, an ML/LLM concepts round and a capstone defence cross-examining your leakage, eval and architecture choices; the coding round is fed by a timed DSA drill on arrays, strings, hashmaps, two pointers and simple recursion in Python, closing the light interview strand carried as self-study across Weeks 9–12 (junior screening rounds only, 4–6 hours total). Scope: Core, covered in full.
Demo Day & Job-Search Plan	The deployed capstone presented with Q&A, then a target-company list (Nepal plus remote), an application tracker, a weekly outreach target, a referral script and a follow-up cadence. Scope: Hands-on.

Weekly revision (last hour): Review of the live deployment (FastAPI, Docker, tracing, cost-per-request), the finalised capstone and scorecard, README/demo, resume/LinkedIn, the system-design and mock rounds, and the closing DSA drill; Q&A; demo day and final outcomes.

Phase 4 Final Project: Production-Ready AI System

Description: Ship the capstone as a live, production-ready AI service with FastAPI, Docker, per-request tracing and a real cost figure, plus a portfolio-grade writeup that stands up in recruiter and client conversations.

Requirements:

- A FastAPI service containerised with Docker and deployed live at a public URL (the guided GPU-cloud deploy to RunPod/Lambda as time allows; any publicly reachable deployment qualifies)
- Per-request tracing (latency, tokens, cost), at least one alerting rule, and a real cost-per-request figure stated in the README
- Deeper MLOps (MLflow tracking, a CI eval gate, an AI-Gateway cost dashboard) layered on as time allows
- A polished repo (README, architecture diagram, eval scorecard, demo video) plus a completed mock interview and 90-day plan

PROJECT POOL

Each student picks **one** capstone with their mentor in the Week 8 architecture-review slot. Pool rotates per batch:

#	PROJECT	STACK FOCUS
1	Production RAG over private docs (evaluated with RAGAS)	Embeddings, Pinecone, retrieval evaluation
2	Multi-modal AI assistant (text + image inputs via Claude Vision / OpenAI vision (ChatGPT) / Gemini vision)	Multi-modal APIs, prompt engineering
3	Domain-tuned agent with tool use and LangGraph workflows	LangGraph, LangSmith, eval
4	Cost-optimised LLM service with caching + batch processing	Token optimisation, Anthropic/OpenAI Batch APIs
5	Fine-tuned local model deployed with Ollama + FastAPI	LoRA, quantisation, serving
6	ML pipeline with cost monitoring + rollback	MLflow, FastAPI, Docker, spend tracking

TECHNICAL COMPETENCIES ACHIEVED

Python & Classical ML

- Write production Python with typed functions, async and clean project structure
- Wrangle tabular data with numpy and pandas using vectorised operations
- Frame problems as supervised or unsupervised and diagnose overfitting
- Train and interpret linear regression, logistic regression, decision trees and random forests
- Report classification (precision, recall, F1, ROC-AUC) and regression (RMSE, MAE, R^2) metrics
- Engineer features and re-evaluate models with discipline

Deep Learning

- Explain neural networks, layers, activations and backpropagation conceptually
- Implement a PyTorch training loop on toy data with autograd, loss, optimizer and GPU-aware patterns
- Reason about attention, transformer architecture, tokenisation and embeddings
- Read and run a HuggingFace fine-tuning notebook (previewed in Week 5; the hands-on LoRA/QLoRA build comes in Week 11)

LLM + RAG

- Use multi-provider LLM APIs (OpenAI, Anthropic, Google Gemini, Groq) with streaming, structured outputs (strict mode) and reasoning models (ChatGPT Thinking, Claude Extended Thinking, Gemini thinking, DeepSeek R1 / V-series)
- Front the providers with an AI Gateway (OpenRouter, Portkey, Helicone) for routing, retries and spend control
- Build function-calling pipelines that return pydantic-validated JSON
- Embed with modern models (text-embedding-3, voyage-4, Cohere Embed 4, Gemini Embedding, bge-m3, Qwen3-Embedding, Matryoshka), chunk and hybrid-search corpora (Pinecone, ChromaDB, pgvector), choosing the embedding model against a small labelled

retrieval set

- Add a reranker (Cohere Rerank, bge-reranker) for two-stage retrieval
- Practise context engineering: curate the system prompt, tools, retrieved context, memory and history that fill a finite window
- Compose RAG pipelines in LangChain (LCEL, with LangGraph for stateful flows) and LlamaIndex
- Improve retrieval with multi-query rewriting and parent-document retrieval; recognise when agentic RAG or GraphRAG (Microsoft GraphRAG, LightRAG, LazyGraphRAG) is the right next step and how to route by cost
- Evaluate RAG with RAGAS (faithfulness, answer relevancy) and debug retrieval issues

Agents

- Explain and hand-build a minimal agent harness, the agentic loop (gather context → act → verify → repeat), tool dispatch and the model-decides/harness-does split, and tell a harness from a framework from a runtime
- Implement ReAct loops and define pydantic-validated tool schemas
- Build and consume MCP (Model Context Protocol) servers, now an open standard under the Linux Foundation, so tools plug into Claude Desktop, Cursor, ChatGPT and other MCP-aware clients, and place MCP, A2A and AG-UI in the agenttools / agentagent / agentuser protocol stack
- Build tool-using agents with LangChain's `create_agent` on the LangGraph engine (knowing the older AgentExecutor and langgraph.prebuilt `create_react_agent` are now legacy/maintenance), with retries and memory
- Give agents persistent memory with a memory layer (Mem0 hands-on; Zep + Graphiti temporal knowledge graph, Letta / MemGPT and LangMem at awareness)
- Apply agent context-engineering techniques: compaction, note-taking memory, sub-agent isolation, just-in-time retrieval, tool-result clearing
- Build a deep agent with `deepagents` (a planning todo tool, isolated sub-agents, a virtual file system and a detailed system prompt) and choose it over a plain ReAct agent for long-horizon, planning-heavy work
- Orchestrate multi-step workflows with LangGraph StateGraphs, conditional edges and supervisors
- Compare LangGraph against the OpenAI Agents SDK and the Claude Agent SDK, distinguishing harness from framework from runtime, and choose with reason (CrewAI and AutoGen at awareness)
- Reason at awareness level about Computer Use / browser-driving agents (Anthropic Computer Use, Browserbase, Playwright)
- Add human-in-the-loop approval gates and checkpointing
- Instrument agents with LangSmith (or Langfuse / Helicone / Arize Phoenix) for observability and evaluation

ML

- Track experiments and manage model lifecycle with MLflow

- Wire GitHub Actions to run RAGAS / LLM-as-judge evals on every pull request
- Serve models with FastAPI and pydantic schemas, containerise with Docker
- Deploy to GPU clouds (RunPod, Lambda) and verify latency
- Add AI-specific monitoring (latency, quality drift), alerts and caching
- Adapt models cheaply with LoRA/QLoRA and HuggingFace PEFT, bootstrapping data with synthetic generation when no labels exist
- Run local inference with Ollama (GGUF Q4/Q5 quantisation, with llama.cpp under the hood); know vLLM and SGLang as the production self-host step up (HuggingFace TGI is now legacy)
- Run evaluation with Promptfoo, DeepEval, Braintrust or OpenAI Evals alongside RAGAS and LangSmith, tracing over the OpenTelemetry GenAI standard
- Write model cards, run red-teaming and gate deploys on evaluation scores

SOFT SKILLS DEVELOPED

- Reading arXiv papers at awareness level to keep up with the field
- Explaining attention, embeddings and hallucination in plain language
- Defending framework choices (LangChain vs LlamaIndex, LangGraph vs OpenAI Agents SDK vs Claude Agent SDK, Mem0 vs Zep) with evidence, and telling a harness from a framework from a runtime
- Writing RAG and agent evaluation reports that non-engineers can act on
- Running end-to-end tests and architecture reviews with peers
- Walking through ML system design questions end-to-end
- Using AI coding assistants (Copilot, Cursor, ChatGPT) responsibly, reading and verifying every generated line
- Reading primary model, API and spec docs (Anthropic, OpenAI, the MCP spec) directly as they ship, because best practice in this field changes quarterly
- Communicating in clear written English with proactive, async status updates, the hard gate for landing remote and contract roles from Nepal

JOB READINESS ASSESSMENT

A graduate is considered job-ready for junior AI engineering roles when they meet all of:

- Minimum 80% live-class attendance across the 12 weeks
- A deployed capstone AI system on the public internet with a live URL
- A GitHub profile with production-grade AI code (RAG, agents, ML)
- Demonstrated RAGAS or LLM-as-judge evaluation on the capstone
- LangGraph agent with tools, a memory layer (Mem0), at least one MCP-served tool and LangSmith traces
- At least one fine-tuning run (OpenAI fine-tune or LoRA/QLoRA PEFT)
- The free Hugging Face Agents Course Certificate of Fundamentals linked on the GitHub or LinkedIn profile
- FastAPI + Docker deployment with per-request tracing, latency and drift monitoring, and a stated cost-per-request in the README (the guided GPU-cloud deploy where time allowed)

- Passing score on a mock ML system design interview
- Rewritten resume and LinkedIn leading with capstone and evaluation metrics

ASSESSMENTS

1. Saarathi Gate Assessment (pre-course, diagnostic)

Run before Day 1. Aptitude diagnostic with no pass or fail. We use it to map your starting point, learning style and pace so trainers can tune support from day one.

2. Saarathi Certification Exam (end-of-course)

Sat in the final week of the course. 2 hours, around 30 questions specific to this course. Mixed formats are used: single choice, multi-select, true/false, numeric, ordering, matching, fill-in-the-blank, short and long answer, code questions where the track is technical, plus short case studies.

- Pass mark: 60 percent
- Mandatory for the Saarathi Certificate of Completion
- Invigilated by a Saarathi mentor
- Camera proctored: your webcam must be on before the timer starts and stay on for the full two hours; stills are saved for your trainer to review
- Questions can be answered in any order and revisited, but every question needs an answer before you can submit

Recommendation letters are issued by mentors only to learners who pass the Certification Exam, and only when the mentor is willing to vouch for the learner in writing. A pass is necessary but not sufficient, and a letter is never guaranteed.

CERTIFICATION REQUIREMENTS

Students receive the Saarathi Academy AI Engineering & Machine Learning Completion Certificate when they meet all of:

- Minimum 80% attendance across 12 weeks of live classes
- Submission of all 12 weekly deliverables reviewed by a trainer
- A deployed capstone AI system reachable at a public URL
- A passing grade on the Week 12 mock ML system design interview
- A delivered demo-day presentation with Q&A
- A public writeup on the capstone's RAG or agent evaluation
- A pass in the Saarathi Certification Exam, as described above

RECOMMENDED BOOKS & RESOURCES

Books

- "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron
- "Designing Machine Learning Systems" by Chip Huyen
- "Deep Learning with PyTorch" by Eli Stevens, Luca Antiga and Thomas Viehmann
- "Natural Language Processing with Transformers" by Lewis Tunstall, Leandro von Werra and Thomas Wolf
- "Machine Learning Engineering" by Andriy Burkov

Official Docs & Learning Resources

- Python docs at docs.python.org and PyTorch docs at pytorch.org/docs
- scikit-learn documentation at scikit-learn.org
- HuggingFace Transformers and PEFT at huggingface.co/docs
- OpenAI API reference at platform.openai.com/docs
- Anthropic API documentation at docs.anthropic.com
- LangChain 1.0 docs at docs.langchain.com and LangGraph docs at langchain-ai.github.io/langgraph
- LlamaIndex at docs.llamaindex.ai
- RAGAS at docs.ragas.io, DeepEval at docs.confident-ai.com and MLflow at mlflow.org/docs
- Anthropic "Effective context engineering for AI agents" (anthropic.com/engineering), the canonical context-engineering reference
- Model Context Protocol at modelcontextprotocol.io; Mem0 at docs.mem0.ai; Zep / Graphiti at help.getzep.com and github.com/getzep/graphiti
- Ollama at ollama.com/library and llama.cpp GitHub
- Courses: Andrew Ng ML Specialization, Andrej Karpathy "Neural Networks: Zero to Hero", Hugging Face Agents Course (free and certified; its Certificate of Fundamentals is the graded Week 9–10 milestone in this course)

FINAL OUTCOMES

By Week 12 you will be able to:

- Write production Python for AI workflows
- Understand classical ML and pick the right model for the job
- Reason about transformer architecture well enough to debug LLM behaviour
- Pick the right LLM for the job across fast, reasoning (ChatGPT Thinking, Claude Extended Thinking, Gemini thinking, DeepSeek R1 / V-series) and multi-provider setups, optionally fronted by an AI Gateway
- Practise context engineering across prompts, RAG and agent memory to keep systems inside a finite context window
- Build production RAG pipelines with modern embeddings, hybrid retrieval, reranking and RAGAS / Promptfoo / DeepEval / Braintrust evaluation, knowing when to reach for agentic or graph RAG
- Design tool-using agents with LangGraph, a memory layer (Mem0) and MCP-served tools, understand the agent harness around the model and reach for a Deep Agent on long-horizon work, while being able to defend that choice against the OpenAI Agents SDK, the Claude Agent SDK and CrewAI
- Fine-tune models with LoRA/QLoRA (including synthetic data bootstrapping) and run local inference with Ollama, with awareness of vLLM/SGLang for production
- Deploy AI systems with FastAPI + Docker, monitor them through LangSmith / Langfuse / Helicone / Arize Phoenix, and operate them with per-feature spend dashboards

CAREER PATHS & TRAJECTORY

ROLE	TIMELINE	FOCUS
AI Engineering Apprentice / Junior LLM-App Developer	First 3–6 months	Ship LLM, RAG and agent features under review; convert the portfolio into a first junior role
Junior AI Engineer / Applied ML Engineer	0–2 yrs	Build LLM features, RAG, agents
AI Engineer / ML Engineer	2–4 yrs	Own AI systems in production
Senior AI Engineer / ML Platform Engineer	4–7 yrs	Architecture, platform, cost/reliability
AI Solutions Architect / Applied AI Lead	7+ yrs	Strategy, org-wide impact

ENROLLMENT

- Next Batch: see the live course page
- Location: Old Baneshwor Chowk, Kathmandu, Nepal
- Mode: Online + Offline
- Contact: 9744442469, 9761095364
- Limited to 10 seats per batch

AI Engineering & Machine Learning — full details, fees and next batch dates:
saarathiacademy.com.np/ai-engineering-and-machine-learning-course-in-nepal

Version 1.0 · Effective June 25, 2026 · Review December 25, 2026 · Official Academic Document